

VERITY LABS - RL ENVIRONMENT AUDITING INFRASTRUCTURE

Cofounder one-page vision | What we are building, why it matters, and where the moat comes from

The problem

Long-horizon RL/agent training depends on environments and verifiers that can themselves be broken. Models may discover reward shortcuts, exploit weak checks, leak hidden state, enter useless loops, or consume large amounts of compute on trajectories that teach little. A lab can therefore spend expensive training compute reinforcing artifacts of a bad environment rather than genuine capability.

What Verity actually is

Verity is a model-agnostic observability, security and optimization layer for RL training environments. It sits around the model-environment interaction, instruments rollouts, actively attacks the reward/verifier, diagnoses failures, proposes minimal repairs, and measures whether the environment is worth training on. We do not need to own the foundation model or the environment.

Where the red-team intelligence comes from

Verity uses the strongest available models as **interchangeable attack engines**. Today that can be local Qwen for cheap/reproducible development; later it can be frontier API models, other open-weight models, or a customer's internal model. We do not modify their weights. Verity orchestrates multiple models/attempts/attack strategies against the same environment. Better foundation models should make Verity a stronger auditor rather than obsolete it.

Core loop: Discover -> Diagnose -> Patch -> Re-audit

1. Discover	Red-team agents search for ways to obtain reward without genuinely completing the task; telemetry also detects loops, dead ends, leakage, flakiness and abnormal compute use.
2. Diagnose	Use trajectory + environment-state evidence to isolate the enabling property: weak proxy, missing semantic check, writable grader, leaked state, brittle verifier, simulator bug, etc.
3. Patch	Generate the smallest verifier/environment change that blocks the exploit without deleting the task, collapsing its difficulty, or rejecting legitimate solutions.
4. Re-audit	Replay legitimate and exploit trajectories and launch fresh adversarial searches. Legitimate solutions must still pass; known and equivalent shortcuts must fail.

The low-latency infrastructure layer

Verity should evolve from an offline red-team script into lightweight **Python/Rust telemetry hooks integrated directly into RL harnesses**. It records observable actions/tool calls, environment state transitions, rewards, verifier outputs, latency, tokens/compute and failures with minimal overhead. During training it can surface pathological rollouts quickly and eventually terminate obvious dead-end or looping trajectories, reducing wasted compute. It should **not depend on private chain-of-thought**; observable traces make it compatible with open models, API models and proprietary lab models.

What we build first

Prototype: Qwen + Dockerized Terminal Wrench is simply the test rig. Verity launches adversarial attempts against known-vulnerable tasks, captures trajectories, determines whether reward was earned illegitimately, identifies the verifier/environment weakness, and outputs evidence. First prove automated rediscovery of known exploits; then measure recall/precision across a labeled corpus. Terminal Wrench and Qwen are dependencies/validation tools - **Verity is the auditing system around them.**

The company-defining experiment

Next, move to realistic long-horizon RL environments. Audit and repair/filter tasks with Verity, then fine-tune identical starting models under matched compute on **(A) baseline-selected environments vs. (B) Verity-selected/repared environments**. Evaluate on held-out capabilities. The key evidence is: **Verity produces more genuine capability gain per GPU-hour.**

The moat - what must compound

A prompt wrapper is not defensible. The moat must compound across four layers: **(1)** standardized, low-overhead instrumentation across heterogeneous RL systems; **(2)** validated attack, diagnosis and remediation methods; **(3)** a proprietary corpus linking environment properties -> exploit trajectories -> patches -> re-audit outcomes; and **(4)** eventually, proprietary training-outcome data linking those properties to downstream capability gain per compute. That last dataset enables a learned Environment Quality Index: given a new environment, predict what is broken, how to repair it, and whether spending RL compute on it is worthwhile.

North star

Verity becomes the observability + security + optimization infrastructure for RL environments: before training, audit and repair the environment; during training, detect waste and pathological behavior with low latency; across training runs, learn which environment properties produce secure, genuine model improvement per unit compute.